

**LAPORAN PRAKTIKUM
TEKNOLOGI IOT II**



DI SUSUN OLEH :

NAMA : DEA ZUL HAFNI SHERLYZA

NPM : 25782065

KELAS : TRI 3 C

**PRODI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026**

Modul 4: Komunikasi Asinkron dan WebSockets

Tujuan Praktikum

- Memahami konsep komunikasi dua arah secara *real-time* antara perangkat mikrokontroler (NodeMCU ESP8266) dengan antarmuka web.
- Mengimplementasikan pembacaan sensor suhu dan kelembapan (DHT11/DHT22) serta pengendalian aktuator (LED) secara fisik menggunakan push button maupun secara digital melalui web dashboard.

4.4.2 Instalasi Pustaka Khusus (*Manual via GitHub*)

Karena berbasis teknologi Asinkron pihak ketiga, Anda harus mengunduh kode sumber pustaka ini secara manual dalam format .zip dari GitHub, lalu memasangnya ke Arduino IDE via menu Sketch > Include Library > Add .ZIP Library....

Silakan kunjungi tautan berikut dan unduh berkas ZIP dari menu Code > Download ZIP:

1. ESPAsyncWebServer: <https://github.com/ESP32Async/ESPAsyncWebServer>
2. ESPAsyncTCP: <https://github.com/ESP32Async/ESPAsyncTCP>

(Perhatian: Jika Anda menggunakan mikrokontroler seri ESP32 di luar praktikum ini, pustaka TCP-nya bernama AsyncTCP).

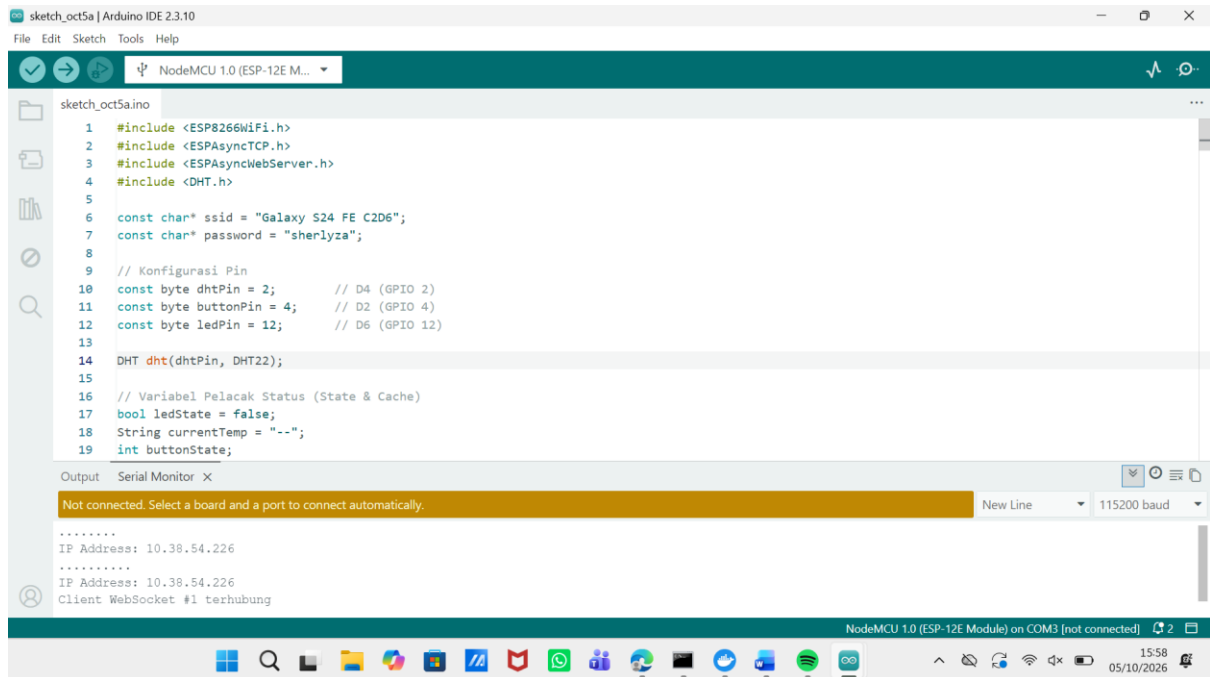
4.5 Praktikum

4.5.1 Praktikum 1: Real-Time Dashboard & Sinkronisasi Tombol Fisik

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (push button) di breadboard, ATAU melalui tombol di antarmuka web smartphone Anda. Keduanya harus tersinkronisasi dua arah secara real-time. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka smartphone harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

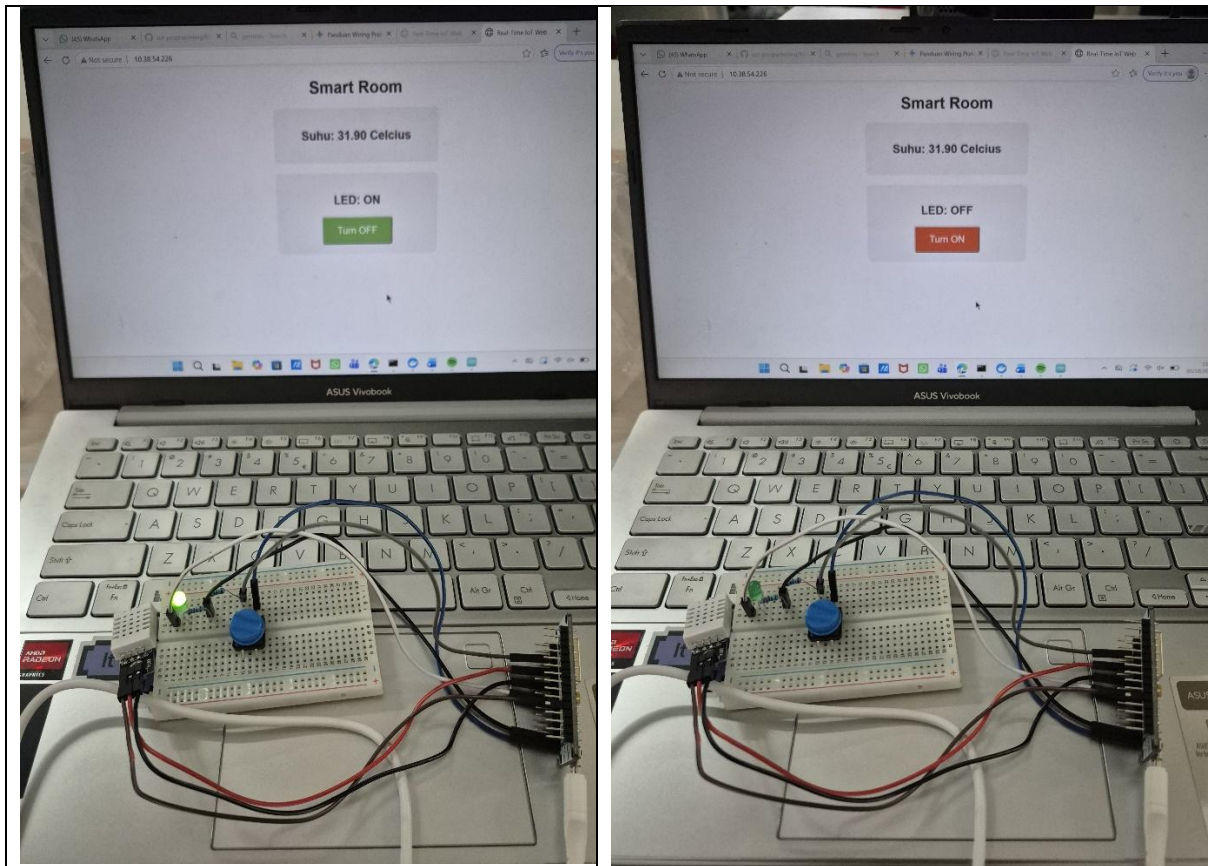
Langkah Kerja Rangkaian:

1. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU (sama seperti rangkaian LED pada Modul 3).
2. Hubungkan pin DATA / OUT DHT ke pin D4 (GPIO 2). (VCC DHT ke 3V3, GND ke GND).
3. Pasang Push Button pada breadboard, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur 3V3 (3.3V). Hubungkan kaki seberang tombol ke pin D2 (GPIO 4) NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki Resistor 10k Ohm, lalu hubungkan ujung kaki resistor lainnya ke jalur GND (konfigurasi resistor pull-down eksternal seperti pada Modul 1).



Berdasarkan pengujian yang telah dilakukan, sistem berhasil melakukan sinkronisasi dua arah (two-way synchronization). Ketika tombol fisik pada breadboard ditekan, status LED dan tampilan pada dashboard web berubah secara seketika (real-time). Begitu pula sebaliknya, ketika tombol pada antarmuka web ditekan, status nyala lampu LED dan pembacaan sensor berfungsi dengan baik tanpa adanya jeda yang signifikan

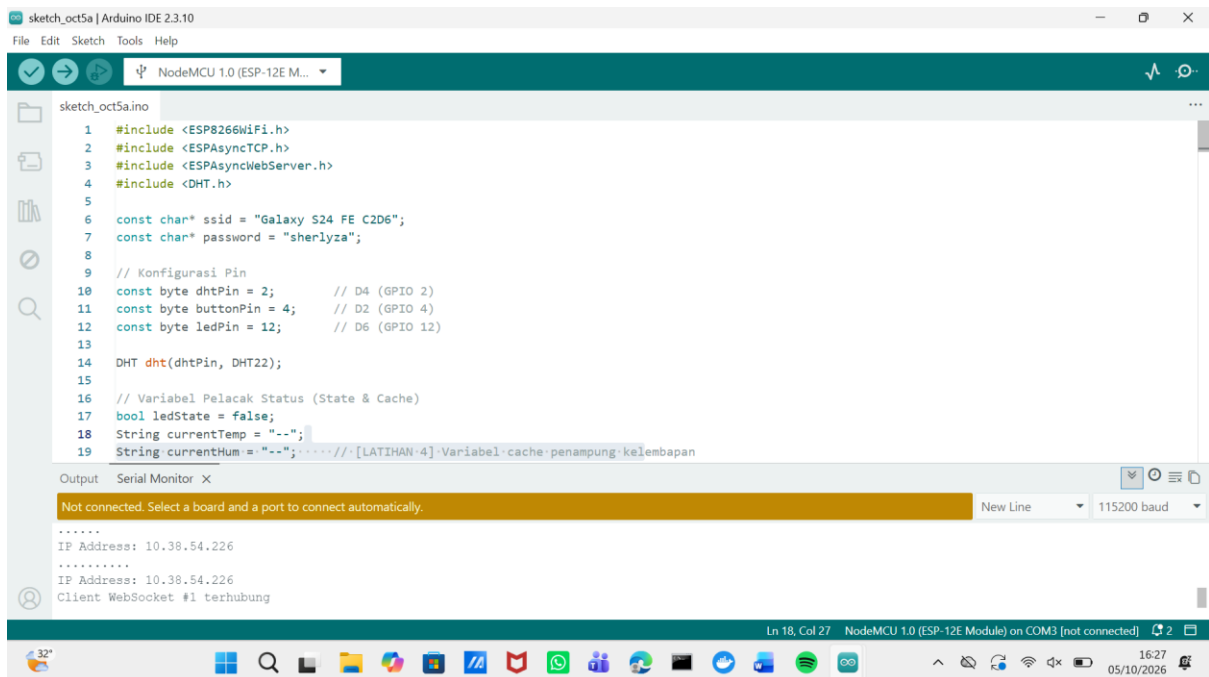
Gambar hasil praktikum :



4.6 Latihan

- Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?
- Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?
- Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?
- Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah **program** agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via *WebSockets*:

- i. Sisi Back-End (C++): Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
- ii. Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!



```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "Galaxy S24 FE C2D6";
7 const char* password = "sherlyza";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 String currentHum = "--"; // [LATIHAN 4] Variabel cache penampung kelembapan
```

Output Serial Monitor X

Not connected. Select a board and a port to connect automatically.

IP Address: 10.38.54.226

IP Address: 10.38.54.226

Client WebSocket #1 terhubung

Ln 18, Col 27 NodeMCU 1.0 (ESP-12E Module) on COM3 [not connected] 16:27 05/10/2026

Hasil latihan :

